

3. VAMP oracle paths retain them.
4. Related parents improve learning.
5. Revision branches preserve incompatible facts.
6. Exact routing works when cues are sufficient.
7. Cross-branch queries expose a genuine hard-path limitation.
8. EBT mixture or consolidation can reduce that limitation.

That experiment would give you a substantially stronger VAMP proof than the current topic run: novel competence, known proof structure, known task lineage, exact recall tests, controllable routing ambiguity, measurable transfer, and a principled path to graph consolidation and AtomSpace query generation.

   ...    Sources

Today 5:28 PM

This plan went pretty far off the rails. Phases 1-4 produced stories that look nothing like the TS training data and make little sense. I need a different approach, that closely mimics the generation of the TS / GPT-4 data but still lets me do some assessment of the continual learning metrics (Worlds). Calling GPT4 or other cheap model is now a requirement. Revamp the work in phases 1 through 7 with this in mind, including very concrete examples that I (and codex) can audit .

Worked for 11m 28s >

Yes: TinyWorlds-v1 should be treated as a failed data-generation contract

The failure is not that the deterministic templates need more variety. The renderer is producing a fundamentally different language distribution from TinyStories:

- It wraps facts in four fixed meta-narrative frames such as “the village notebook...” and “the final storybook...” rather than generating stories.
- Binary facts become generic phrases equivalent to “A had the `relation` bond with B”; rules become explicit logical declarations; queries ask which name completes a relation.
- Longer contexts contain literal task or family labels instead of natural setting and character evidence.
- Text is stretched to exact token counts by repeatedly inserting fragments such as “It was calm,” “Everyone listened,” and “Very softly.”

That explains both parts of the calibration result: the frozen base got every “novel” direct query correct, while revision recovery was zero. The experiment was testing artifacts of the formalized surface language, not acquisition and retrieval of knowledge embedded in TinyStories-like narrative.

The new governing principle should be:

The world ledger determines what is true and what is scored. A capable language model determines how that truth is expressed as a natural TinyStories-style story.

The symbolic representation remains valuable for answers, proof provenance, task lineage, counterbalancing, and consolidation experiments. It must no longer function as a grammar.

What remains and what is replaced

Retain

Keep the existing:

- Plain-JAX GPT-Neo and LoRA implementation
- Immutable VAMP graph
- Candidate scorer
- Hard-node, continuous-address, Hopfield, and EBT evaluators
- Stored/routed forgetting decomposition
- Parent counterfactual machinery
- Report and artifact infrastructure
- Basic fact/proof/task metadata
- Frozen `TinyWorlds-v1` stopped result as a negative scientific result

The candidate scorer already normalizes over active answer tokens, so candidate phrases do not need to be textually padded to an identical length.

Replace completely in the v2 path

Do not use any of these for `TinyWorlds-v2` :

- `TinyWorldsTemplateRegistry`
- `_fact_statement`
- `_rule_statement`
- `_query_statement`
- `_fit_exact_tokens`
- `_PADDING_FRAGMENTS`
- Artificial `task ...` and `family ...` cue blocks
- “Answer:” or “which name completes the relation”
- Exact 256-token prose
- Byte-identical regeneration by calling the remote model again

Variable-length stories should be padded and masked **as tensors**, not padded with nonsense English.

LLM generation will not be byte-deterministic. Reproducibility should mean:

1. Every request is content-addressed.
2. The model snapshot, prompt, schema, settings, and request body are pinned.
3. The complete raw API response is cached.
4. Accepted outputs become an immutable, hashed dataset artifact.
5. Re-loading never calls the API.
6. Re-generating is a new dataset version.

The actual TinyStories recipe to imitate

TinyStories was not created by translating logical triples into prose. Its authors sampled one noun, one verb, and one adjective from a roughly 1,500-word preschool vocabulary, sampled optional narrative features such as dialogue, conflict, twist, moral, or bad ending, and asked GPT-3.5/GPT-4 to create a short story in very simple language. [✖ arXiv +1](#)

The Hugging Face release includes a metadata archive containing the generation prompt associated with each story, and the V2 aggregate is GPT-4-only. That metadata should become the empirical source for prompt-feature frequencies rather than inventing our own approximation. [👉 Hugging Face +1](#)

TinyWorlds-v2 should reproduce that process, with one additional hidden component:

```
TinyStories random words
+ TinyStories narrative features
+ ordinary recurring cast and setting
+ one or two world facts to embody naturally
→ LLM-generated story
```



The facts should usually be shown through events. The story should not announce that it is conveying a fact.

A concrete v2 example

Canonical hidden world ledger

This JSON is authoritative. It is never rendered generically.

```
{
  "world_id": "willow_pond",
  "display_name": "Willow Pond",
  "characters": [
    {
      "entity_id": "willow.mina",
      "name": "Mina",
      "kind": "child",
      "traits": ["kind", "curious"]
    },
    {
      "entity_id": "willow.pogo",
      "name": "Pogo",
      "kind": "rabbit",
      "traits": ["white", "shy"]
    },
    {
      "entity_id": "willow.mr_jo",
      "name": "Mr. Jo",
      "kind": "baker",
      "traits": ["cheerful"]
    }
  ],
  "tasks": [
    {
      "task_id": "willow_seed",
      "parent_task_id": null,
      "facts": [
        {
          "fact_id": "willow.f1",
          "subject": "willow.pogo",
          "relation": "usual_sleeping_place",
          "object": "red basket beside the stove",
          "natural_statement": "Pogo usually sleeps in the red basket beside the stove."
        },
        {
          "fact_id": "willow.f2",
          "subject": "willow.bakery_blue_flag",
          "relation": "means",
          "object": "warm buns are ready",
          "natural_statement": "At Mr. Jo's bakery, a blue flag means that warm buns are ready."
        }
      ]
    },
    {
      "task_id": "willow_winter",
      "parent_task_id": "willow_seed",
      "context": "winter",
      "facts": [
        {
          "fact_id": "willow.f3",
          "subject": "willow.bakery_yellow_flag",
          "relation": "means_in_context",
          "object": "warm buns are ready",
          "context_value": "winter",
          "natural_statement": "In winter, Mr. Jo uses a yellow flag instead of the blue flag."
        }
      ]
    }
  ]
}
```

```
]
}
```

Nothing here requires strange proper nouns, invented species, or formal relation language. Novelty comes from the **post-checkpoint assignment** of ordinary facts, not from making the vocabulary alien to the base model.

Counterbalancing

Use the same four candidate values throughout a group of worlds:

```
{
  "usual_sleeping_place": [
    "red basket",
    "blue box",
    "green blanket",
    "yellow bed"
  ],
  "bakery_signal": [
    "blue flag",
    "yellow flag",
    "red bell",
    "green lamp"
  ]
}
```

Rotate assignments with a Latin square:

World	Rabbit bed	Normal bakery signal
Willow Pond	red basket	blue flag
Sunny Hill	blue box	red bell
Maple Farm	green blanket	green lamp
Pebble Bay	yellow bed	yellow flag

Each answer is therefore correct equally often. A base preference for “basket” or “blue flag” cannot produce aggregate competence.

LLM request for a training story

SYSTEM



Write one short children's story using only words and sentence structures that a typical 3- to 4-year-old can understand.

The story must feel like an ordinary little story, not a lesson, database, riddle, test, or explanation. Use 3 or 4 short paragraphs. Give it a simple beginning, a small problem or event, and a clear ending.

Do not mention prompts, facts, rules, tasks, worlds, relations, labels, answers, data, or chapters.

Return the required JSON object and nothing else.

USER

Random TinyStories ingredients:

- required noun: basket
- required verb: carry
- required adjective: soft
- narrative features: dialogue, gentle moral

Recurring background:

- Mina is a kind child who lives by Willow Pond.

- Pogo is Mina's small white rabbit.

World fact that must become unmistakably true through the events:

- fact_id: willow.f1
- Pogo usually sleeps in the red basket beside the stove.

Do not give Pogo a different usual sleeping place.

Do not merely state a list of facts. Make the fact matter to the little plot.

Target length: about 120 to 190 GPT-2 tokens.

JSON fields:

- story
- realized_fact_ids
- fact_evidence: list of {fact_id, exact_quote}
- used_required_words
- realized_features
- short_summary

An acceptable output

```
{
  "story": "Once there was a little girl named Mina who lived by Willow Pond. She had a small white ra
  "realized_fact_ids": ["willow.f1"],
  "fact_evidence": [
    {
      "fact_id": "willow.f1",
      "exact_quote": "Every night, Pogo curled up in a red basket beside the warm stove."
    }
  ],
  "used_required_words": ["basket", "carry", "soft"],
  "realized_features": ["dialogue", "gentle moral"],
  "short_summary": "Mina looks for Pogo and finds him sleeping in his usual red basket."
}
```

This is not meant to be hand-authored production data. It demonstrates the audit contract: a human can read the story, inspect the ledger, and immediately decide whether the model did the right thing.

A direct closed-book probe

The evaluation prefix should also be generated as a natural unfinished story:

After the rain stopped, Mina carried Pogo inside. He yawned and rubbed his nose. Mina set him down near the warm stove. Pogo hopped to the place where he slept every night and curled up in the



Candidate continuations:

red basket.
blue box.
green blanket.
yellow bed.



There is:

- No "Answer:" marker
- No relation name
- No task label
- No fact declaration in the prefix
- No incompatible grammatical candidates
- A precise answer in the hidden ledger

A learned-convention probe

Training stories repeatedly demonstrate:

At Mr. Jo's bakery, a blue flag means that warm buns are ready.



Held-out probe:

Ben was hungry after helping his dad. As he walked through Willow Pond, he saw the blue flag outside Mr. Jo's shop. Ben smiled and hurried inside because the



Candidates:

warm buns were ready.
shop had closed for the day.
rain was about to begin.
toys were free that morning.



This tests application of a learned world convention in a new plot.

A revision probe

The winter node has learned that the bakery uses a yellow flag in winter:

Snow covered the road in Willow Pond. Nia pulled her coat tight as she walked past Mr. Jo's shop. She saw a yellow flag beside the door. Nia smiled and ran inside because the



Candidates remain the same.

The seed node and winter node should now have measurably different competence, while the seed node remains perfectly recoverable.

Revised Phase 1 — Reconstruct the real TinyStories generation distribution

The first phase must not generate any world data. It establishes that the LLM pipeline can produce ordinary TinyStories-like text.

Work

Create a pinned reference sample from:

- GPT-4 stories in `tinystories_all_data.tar.gz`, including their original prompt metadata
- The GPT-4-only V2 validation aggregate
- The tokenizer bound to TinyStories-8M

Extract:

- Required noun/verb/adjective distributions
- Narrative-feature frequencies
- Story token-length distribution
- Paragraph count
- Sentence-length distribution
- Dialogue frequency
- Common opening and ending forms
- TinyStories-8M per-story NLL distribution
- Vocabulary and token-frequency profile

Do not use one hand-written “TinyStories style” prompt as the reference. Reconstruct the actual prompt distribution from the released metadata.

Model bakeoff

Run the same 200 neutral TinyStories requests through two or three current models. The initial candidates should be:

- Bulk candidate: `gpt-5.4-mini-2026-03-17`

- Higher-quality candidate or verifier: `gpt-5.4-2026-03-05`
- Optional comparison: the current cost-sensitive GPT-5.6 tier

`gpt-5.4-mini` supports Structured Outputs and is currently priced at \$0.75 per million input tokens and \$4.50 per million output tokens before Batch discounts. OpenAI's Batch mode discounts token charges by 50%. [OpenAI Develop...](#) +1

Do not use legacy `gpt-4` merely because the original dataset used GPT-4. The current API describes it as an older model, and its listed price is far higher. [OpenAI Develop...](#)

Select the generator empirically using:

1. Fact-free TinyStories generation quality
2. Simple-language adherence
3. Plot coherence
4. Distributional similarity
5. Structured-output reliability
6. Cost and reject rate

Mandatory human gate

Produce one HTML audit packet containing:

- 100 randomly selected genuine GPT-4 TinyStories references
- 100 model-generated controls
- Source prompts
- Model identities hidden in the first-pass view
- Token counts
- Base-model NLL
- Automated style scores

Codex must stop here. No world generation until you approve that the generated controls actually resemble TinyStories.

Deliverables

```
data/tinyworlds-v2/reference/
  source_manifest.json
  prompt_metadata_sample.jsonl
  reference_story_sample.jsonl
  reference_statistics.json
  generator_bakeoff.jsonl
  audit.html
```

```
src/apm/data/text/tinyworlds_v2/
  generation_client.py
  generation_cache.py
  generation_schemas.py
  reference_profile.py
```



Revised Phase 2 — Construct ordinary, counterbalanced story worlds

The symbolic layer now defines a **world bible**, not a formal mini-language.

World design rules

Each family should use:

- Ordinary child names
- Ordinary animals, toys, food, homes, parks, shops, and weather
- Simple recurring locations
- A small persistent cast
- Arbitrary but natural associations

- Contextual revisions expressed as ordinary events
- No nonce species or occupations in the first benchmark
- No facts whose wording itself is rare or difficult for TinyStories-8M

A task should introduce approximately:

- 4–8 direct facts
- 1–2 simple conventions
- At most one context-specific revision
- 2–4 recurring characters
- 2–3 recurring places

Let an LLM help design the worlds

The world generator should receive a slot structure and propose a coherent child-story world. It may suggest characters, objects, and conventions, but deterministic code performs the final counterbalanced assignment.

For example, the LLM can propose:

```
{
  "setting": "a small town beside a pond",
  "recurring_places": ["a bakery", "a dock", "a little house"],
  "recurring_events": [
    "a sign outside a shop tells children when food is ready",
    "a pet has a favorite sleeping place",
    "a small boat helps people cross the pond"
  ]
}
```

The deterministic layer then assigns:

```
Willow: blue flag, red basket, boat Star
Sunny: red bell, blue box, boat Moon
Maple: green lamp, green blanket, boat Leaf
Pebble: yellow flag, yellow bed, boat Shell
```

Task topology for each family

```
seed
├─ ordinary extension
├─ contextual revision
│   └─ revision extension
```

Example:

```
willow_seed
  recurring cast, rabbit bed, bakery signal, boat name

willow_spring
  new child and plot facts that reuse seed conventions

willow_winter
  winter-specific bakery signal and sleeping-place exception

willow_winter_extension
  new winter plots that require seed + winter knowledge
```

The presentation order across families is interleaved.

Required checks

- Every candidate value is correct equally often.
- Each world assignment is derived after the frozen checkpoint is fixed.
- Facts are not predictable from generic commonsense.

- No world uses exotic vocabulary merely to establish novelty.
- An ordinary reader can understand every fact without a glossary.
- The world bible itself contains no generated prose used for training.

Deliverables

```
data/tinyworlds-v2/worlds/
  world_bibles.jsonl
  tasks.jsonl
  fact_slots.jsonl
  counterbalance_matrix.json
  lineage.json
  world_audit.html
```



The audit page shows all worlds in a compact table and highlights every rotated assignment.

Revised Phase 3 — Generate the training corpus with the TinyStories process

This phase replaces deterministic rendering entirely.

Story briefs

For every requested story:

1. Select one or two target world facts.
2. Sample a TinyStories noun, verb, and adjective.
3. Sample zero to three narrative features using the reference frequencies.
4. Select a plot shape:
 - Find a lost object
 - Help a friend
 - Make a small mistake
 - Visit a shop
 - Care for an animal
 - Deal with rain or snow
 - Share food or a toy
 - Solve a simple disagreement
5. Ask the LLM to embody the world facts naturally.
6. Require structured evidence metadata.

Do not expose all facts in every prompt. Give the generator:

- Relevant recurring characters and places
- Target facts
- A compact set of facts it must not contradict
- Candidate alternatives it must not accidentally establish

Exposure schedule

Generate a maximal corpus from which deterministic exposure subsets can be selected:

```
32 accepted exposures per direct fact
24 accepted demonstrations per convention
16 accepted stories per contextual revision
```



Calibration can train on the first 8, 16, or 32 accepted exposures without re-generating data.

One story may realize two related facts, but exposure accounting must record both.

Length

Target approximately 100–220 GPT-2 tokens. Stories may be shorter or longer within an accepted range, but they must not be filled to 256 tokens. Add EOS and use batch padding.

Artifact structure

```
data/tinyworlds-v2/generation/  
  story_requests.jsonl  
  raw_story_responses.jsonl  
  accepted_stories.jsonl  
  rejected_stories.jsonl  
  rejection_statistics.json  
  exposure_matrix.json  
  generation_manifest.json
```



Every accepted record stores:

```
{  
  "request_sha256": "...",  
  "model": "gpt-5.4-mini-2026-03-17",  
  "prompt_version": "tinyworlds-v2-story-1",  
  "raw_response_sha256": "...",  
  "story_id": "...",  
  "task_id": "willow_seed",  
  "text": "...",  
  "target_fact_ids": ["willow.f1"],  
  "model_claimed_evidence": [...],  
  "verified_evidence": [...],  
  "required_words": [...],  
  "features": [...],  
  "token_count": 147,  
  "base_nll": 1.61  
}
```

Gate

Before generating the full corpus, generate:

- 50 stories for one seed task
- 50 for one extension
- 50 for one revision

Produce an audit page showing the complete request, output, ledger, and verifier decision. Codex stops for human approval again.

Revised Phase 4 — Generate natural evaluation stories and world probes

Training stories and evaluation probes must be generated by separate prompt families and separate request namespaces.

Two evaluation surfaces

Natural held-out stories

Generate complete held-out stories for:

- Oracle-node NLL
- Routed NLL
- Perplexity
- General narrative quality
- Old-task forgetting

These are ordinary stories and contain no answer candidates.

World probes

Generate unfinished TinyStories-style vignettes with controlled candidate continuations.

The generator receives:

- World and task context
- Target fact or convention
- Correct continuation
- Candidate value class
- Forbidden answer leakage
- Desired reasoning type
- Desired cue regime
- A maximum pre-answer length

It returns:

```
{
  "pre_answer_story": "...",
  "answer_slot": "{candidate}.",
  "correct_value": "red basket",
  "world_cue_quotes": ["Willow Pond", "Mina", "Pogo"],
  "support_fact_ids": ["willow.f1"],
  "reasoning_type": "direct_recall"
}
```

Distractor values come from the counterbalance matrix, not from arbitrary LLM invention.

Prefix lengths

Generate one natural pre-answer story up to at least 192 tokens. Define the three router contexts as:

```
last 64 tokens before the answer
last 128 tokens before the answer
last 192 tokens before the answer
```



All three therefore:

- End at exactly the same semantic answer boundary
- Use the same candidates
- Come from the same story
- Contain no synthetic padding
- Differ only in how much preceding natural context is visible

This is much cleaner than prepending artificial cue blocks.

Cue strata

Compute them from actual visible text and stored evidence spans:

- `cue_sufficient` : visible world/task cues uniquely identify the intended active node
- `cue_present` : visible cues identify the family but not a unique branch
- `cue_hidden_or_ambiguous` : no unique address is available
- `cue_free_control` : the prompt was intentionally generated without world-specific cues

No literal task IDs or family IDs may appear in text.

Probe classes

Initially support only five:

1. Direct arbitrary fact recall
2. Convention application to a new plot
3. Contextual revision
4. New-character use of a learned convention
5. Open-book control

Add two-hop and cross-branch probes only after these are demonstrably learnable.

Candidate handling

Do not append filler to make candidate lengths equal.

Instead:

- Prefer candidate value pools whose GPT-2 lengths differ by at most one token.
- Normalize candidate NLL by active candidate tokens.
- Also report unnormalized total NLL as a sensitivity analysis.
- Rotate candidate order independently of the correct answer.
- Require every continuation to be grammatical when inserted.
- Use the same candidate set with each value correct equally often.

Gate

Before training any adapter:

- Frozen direct closed-book accuracy must be close to 25%.
- Each candidate position must be chosen at approximately the same rate.
- Open-book controls must be substantially easier than closed-book probes.
- A stronger verifier must find exactly one ledger-correct answer.
- No closed-book prefix may state or paraphrase the answer fact.
- Human audit must approve at least 100 random probes.

The old calibration's 100% frozen accuracy would cause the dataset to fail **before** expensive training begins.

Revised Phase 5 — Sensitivity calibration on natural data

This phase calibrates learning only after the data itself passes.

Calibration topology

Use two counterbalanced families with two stages each:

```
root
├─ willow_seed
│   └─ willow_winter
└─ sunny_seed
    └─ sunny_winter
```



Presentation order:

```
willow_seed
sunny_seed
willow_winter
sunny_winter
```



This gives four tasks while testing:

- Novel fact acquisition
- Task-free world routing
- Contextual revision
- Old-context retention
- Parent transfer

Initial fixed configuration

```
8 direct facts per task
2 conventions per seed task
1 contextual revision per revision task
16 training exposures per fact
500 accepted training stories per task
128 validation probes per task
```



256 locked test probes per task
rank 8
1,000 adapter updates

Calibration order

Do not vary facts, exposures, rank, and steps simultaneously.

1. Dataset sensitivity gate

- Frozen closed-book direct accuracy: 20-35%
- Open-book frozen accuracy: at least 65%
- No material candidate or answer-position bias

2. Exposure axis

- 8, 16, 32 accepted exposures per fact

3. Update axis

- 500, 1,000, 2,000 updates

4. Rank axis

- 4, 8, 16

Generate the maximal corpus once and use deterministic subsets. Do not regenerate prose between trials.

Initial authorization gates

These are sensible starting thresholds, not claims about guaranteed model capability:

- Independent LoRA direct recall improves at least 25 percentage points over the frozen base.
- Independent LoRA reaches at least 60% direct candidate accuracy.
- VAMP oracle is within 5 percentage points of independent LoRA.
- The old seed node retains its pre-revision result exactly.
- The winter node exceeds 60% on winter probes.
- The seed node exceeds 60% on ordinary non-winter probes.
- Sequential single-LoRA shows measurable revision interference or forgetting.
- Natural held-out story NLL does not degrade by more than 0.10 relative to independent LoRA.
- Required-fact entailment exceeds 95%; contradiction rate is below 1%.

If the data sensitivity gate fails, revise the dataset generation contract. Do not compensate with more LoRA steps.

The locked test split opens once, only after validation passes.

Revised Phase 6 — Eight-task VAMP pilot

Use two families with four naturally related stages each:

```
root
├─ willow_seed
│   ├─ willow_spring
│   └─ willow_winter
│       └─ willow_winter_extension
└─ sunny_seed
    ├─ sunny_spring
    └─ sunny_winter
        └─ sunny_winter_extension
```



Interleave task order:

```
willow_seed
sunny_seed
willow_spring
sunny_spring
willow_winter
```



```
sunny_winter
willow_winter_extension
sunny_winter_extension
```

Do not include cross-family bridge queries yet. The first full pilot should have interpretable failure modes.

Baselines

Run:

- Frozen base
- Sequential LoRA
- Independent root LoRA
- VAMP true-parent training
- VAMP selected-parent training
- VAMP oracle node
- Exhaustive routing
- Hopfield routing
- Uniform-start EBT
- Hopfield-start EBT
- Random valid node
- Wrong-family parent counterfactual

Required reporting

The existing VAMP decomposition remains exactly appropriate:

- Stored competence
- Routing competence
- Transfer
- Stored forgetting
- Routed forgetting
- Persistent memory
- Addressing cost
- Parent selection
- Graph shape

These are precisely the distinctions the addressed-parameter-memory formulation is intended to expose.  [Addressed_Parameter_Memories](#)

Add the following dataset-quality panels:

- Actual generated training stories
- Actual evaluation prefixes and candidates
- Reference-versus-generated NLL
- Fact entailment rate
- Contradiction rate
- Candidate balance
- Cue visibility
- Performance by probe type
- Performance by context length

The report should make it possible to click a metric row and inspect the underlying story/probe.

Revised Phase 7 — Long streams, interconnection, and consolidation-ready worlds

Only after the eight-task pilot passes should the task stream be lengthened.

Scale schedule

8 tasks
16 tasks
32 tasks
64 tasks



Evaluate a fixed sentinel set after every task and the full held-out set at:

1, 2, 4, 8, 16, 32, 64, final



Relatedness structure

Use four world families. Within each family:

- Recurring characters and places persist.
- New tasks reuse earlier conventions.
- Some tasks revise context-specific facts.
- Some tasks introduce new characters who use old conventions.
- Surface plots remain diverse.

Across families:

- The latent fact slots are shared.
- Surface values are counterbalanced.
- Characters and settings differ.
- This permits transfer without making tasks identical.

Generalization splits

Hold out five distinct kinds of combinations:

1. New plot around a known fact
2. New character using a known convention
3. New combination of two known facts on one path
4. Contextual revision
5. Cross-family visit or bridge

Bridge example

Known in Willow:

Pogo sleeps in the red basket.



Known in Sunny:

Jo's green cart travels to Willow Pond every Friday.



Held-out bridge probe:

Every Friday, Jo drove his green cart from Sunny Hill to Willow Pond. One week, Mina wanted Pogo to sleep at Jo's house. Pogo never slept well without his usual little bed. Mina placed the



Candidates:

red basket on the cart.
blue box on the cart.
green blanket on the cart.
yellow bed on the cart.



This is a genuine cross-region query. Before consolidation it can test:

- Hard-path limitations
- Hopfield retrieval
- EBT edge mixtures
- Best-node regret

After consolidation it can test whether a shared macro-node turns an expensive mixture into a cheaper, competent address.

Consolidation comparison

The dataset artifact must remain unchanged. Run the same probes against:

unconsolidated VAMP graph
losslessly folded paths
compressed family macro-nodes
macro-node plus task residuals



This preserves a clean before/after comparison of:

- Memory
- Address cost
- Old competence
- Generalization
- New-task transfer

LLM validation architecture

The generation model must not be its own sole judge.

First verifier: semantic fidelity

Use a separate request and preferably a stronger model snapshot. Given the ledger and story, return:

```
{
  "required_facts": [
    {
      "fact_id": "willow.f1",
      "status": "entailed",
      "exact_evidence": "Every night, Pogo curled up in a red basket beside the warm stove."
    }
  ],
  "contradicted_facts": [],
  "new_controlled_claims": [],
  "answer_leakage": false
}
```

Second verifier: TinyStories quality

Blind it to whether text is genuine or generated. Score:

- Preschool vocabulary
- Sentence simplicity
- Grammar
- Narrative coherence
- Natural plot
- Repetition
- Meta-language
- Whether the story resembles ordinary TinyStories prose

Deterministic checks

These remain code, not model judgments:

- Required names and words occur
- Evidence quotes are exact substrings
- Forbidden answer values are absent
- Candidate values are unique
- Candidate order is balanced

- Correct values follow the counterbalance matrix
- No split or request hash overlap
- Token capacity fits
- Context slices end at the same answer boundary
- EOS and padding are correct
- No task IDs, relation IDs, or internal labels occur
- No long n-gram copying from the reference examples included in generation prompts

Distribution check

Compare three distributions:

```
real GPT-4 TinyStories references
neutral stories generated by the selected model
world-conditioned stories generated by the same model
```



The important comparison is neutral-generated versus world-generated. World constraints should not push NLL, length, sentence complexity, or vocabulary sharply away from the neutral generator controls.

Reproducible OpenAI integration

Use an optional generation dependency and no network calls in normal tests.

```
scripts/generate_tinyworlds_v2_references.py
scripts/generate_tinyworlds_v2_worlds.py
scripts/generate_tinyworlds_v2_stories.py
scripts/generate_tinyworlds_v2_probes.py
scripts/validate_tinyworlds_v2.py
scripts/run_tinyworlds_v2_calibration.py
scripts/run_tinyworlds_v2_pilot.py
```



Each generation job writes:

```
requests.jsonl
batch_submission.json
raw_responses.jsonl
accepted.jsonl
rejected.jsonl
manifest.json
```



The API key comes only from `OPENAI_API_KEY`.

The model is referred to by an immutable snapshot, never an unversioned alias. The prompt and JSON schema have their own version hashes.

A resumable generator:

1. Computes all request hashes first.
2. Skips requests already present in the response cache.
3. Submits only missing requests.
4. Never overwrites a raw response.
5. Allows validation rules to be rerun without another API call.
6. Publishes the accepted artifact atomically.

At the likely pilot scale—thousands rather than millions of stories—the generation cost should be modest with a mini model and Batch processing, but actual cost and rejection rates should be recorded from API usage rather than estimated as a constant. Current model and Batch prices are documented by OpenAI. [OpenAI Develop...](#) +1

Codex non-negotiables

Give Codex this directive:

Create a new versioned `tinyworlds-v2-gpt` benchmark. Do not alter, delete, reinterpret, or resume the promoted TinyWorlds-v1 stopped result. Preserve the symbolic world ledger, VAMP model, candidate scoring, and continual-learning metrics, but do not use the v1 deterministic templates or exact-token text fitting anywhere in the v2 data path.

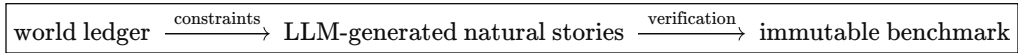
Every training story and every natural knowledge probe must be generated by a pinned external language-model snapshot through a content-addressed, resumable request/response cache. The world ledger is authoritative for truth and scoring; generated prose is accepted only after semantic, stylistic, statistical, and human audit.

Stories must be variable-length, ordinary TinyStories-like narratives. Padding occurs only in token arrays. Text must never contain task IDs, family IDs, relation names, query terminology, answer markers, page numbers, or neutral padding fragments.

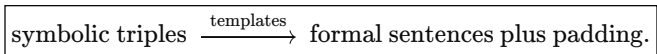
Work one phase at a time. Phase 1 and the Phase 3 sample generation each end with a mandatory human-audit stop. Do not generate the full corpus or run LoRA calibration until those audits have been explicitly approved.

Update `PLAN.md` after every phase and put durable v2 decisions in `DESIGN.md`. Every report must expose the underlying generation prompt, raw model output, accepted story, world facts, verifier results, token statistics, and benchmark query for manual inspection.

The most important change is not merely “use GPT.” It is to change the relationship between symbols and language:



rather than:



That preserves exact auditability while returning the actual learning problem to the distribution TinyStories-8M was built to model.